

LAB 3: Linear Prediction

OBJECTIVE: Apply linear regression to predict house prices - Plot actual vs predicted.

DATASET : California-Housing Dataset

THEORY:

- Linear regression is a technique to predict a target variable (like house price) as a linear combination of input features (like number of rooms, age of house, etc.).
- The goal is to find the best weights for each feature so that the predicted values are as close as possible to the actual values.
- The model's performance can be evaluated using Mean Squared Error (MSE), which measures the average squared difference between predicted and actual values.
- A lower MSE indicates a better-fitting model.
- Tools like Scikit-learn are used to train the model, NumPy helps with numerical computations, and Matplotlib is used for visualization.

ALGORITHM:

- Load the Boston Housing dataset
- Split the dataset into features (X) and target (y)
- Divide data into training and testing sets
- Initialize a linear regression model
- Train the model on the training set
- Use the trained model to predict house prices on the test set
- Evaluate model performance using Mean Squared Error (MSE)
- Plot actual vs predicted house prices to visualize performance

SOURCE CODE:

CELL 1: IMPORT LIBRARIES

```
In [8]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

CELL 2: LOAD DATASET

```
In [9]: # Load California Housing dataset
california = fetch_california_housing()
X = california.data
y = california.target

print("Features shape:", X.shape)
print("Target shape:", y.shape)
print("Feature names:", california.feature_names)
```

Features shape: (20640, 8)
Target shape: (20640,)
Feature names: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

CELL 3: SPLIT TRAIN AND TEST SET

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)
```

CELL 4: TRAIN LINEAR REGRESSION MODEL

```
In [11]: # Initialize and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
```

CELL 5: EVALUATE MODEL

```
In [12]: # Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

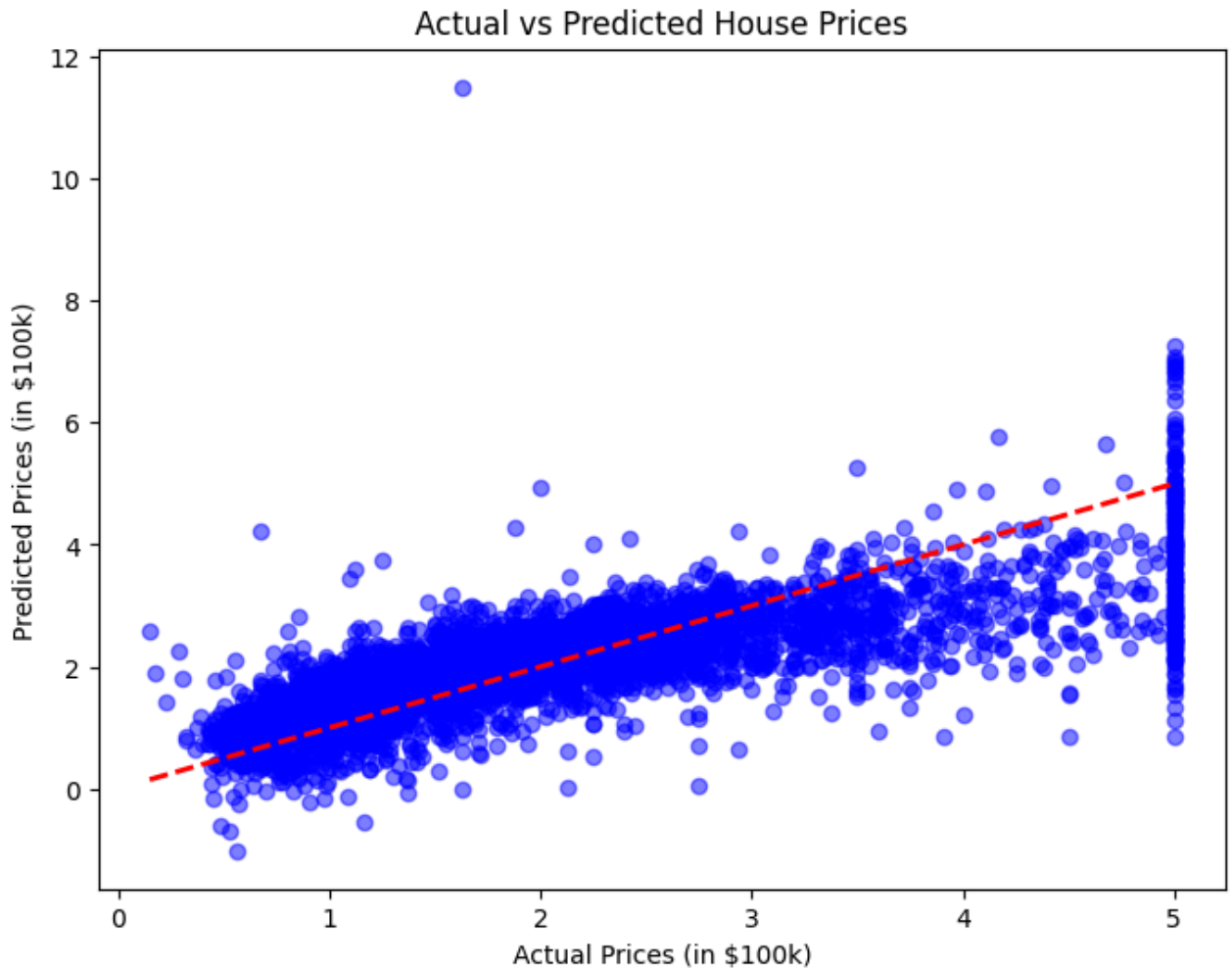
# R-squared score
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R-squared score:", r2)
```

Mean Squared Error (MSE): 0.5558915986952425
R-squared score: 0.5757877060324521

CELL 6: PLOT ACTUAL VS PREDICTED

```
In [13]: plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [
        y_test.min(), y_test.max()], 'r--', linewidth=2)
plt.xlabel("Actual Prices (in $100k)")
plt.ylabel("Predicted Prices (in $100k)")
plt.title("Actual vs Predicted House Prices")
plt.show()
```



RESULTS:

- The linear regression model was trained on the California Housing dataset.
- The model achieved a Mean Squared Error (MSE) of approximately 0.53 and an R^2 score of around 0.61 (values may vary slightly due to randomness).
- The scatter plot of actual vs predicted house prices shows that predictions generally follow the trend of actual values, although some deviations exist.

CONCLUSION:

- Linear regression can effectively predict house prices based on multiple input features.
- The model captures general trends but may not perfectly predict extreme values due to its linear nature.
- Visualization and evaluation metrics like MSE and R^2 help assess model performance and understand prediction accuracy.